AGC Take-Home — Report

Name: Time spent:

Fill in what you got to. Write "not measured" where you did not — that is a fine answer and much better than a guess.

1. What works

Briefly: what is running, and what is not.

Everything is working. TImer driven audio sampling, thru DMA to memory, processing for filter + AGC, DMA back into UART.

Recording looks good, AGC performs as expected, I have concerns about high frequency aliasing into my samples due to lack of a hardware low-pass filter.

2. Timing

	Value
Core clock you configured	170MHz w/ boost mode
How you confirmed it	GPIO toggle at TIM6 interrupt, 8kHz
Worst-case block time (cycles)	126961 (746us @ 170MHz, 9.3% of real time)
As a percentage of the 8 ms budget	9.3%

Where does the time go? Biggest consumer first.

Most CPU time is spent on FIR convolution of the input signal. This is approx 217 floating point operations and some copying and such.

After that there's some more time (about 1/3th as much) on applying linear interpolation for AGC + measuring RMS of recorded blocks.

3. Counters at the end of a full run

Counter	Value
DMA overruns	0

Counter	Value
ADC overruns	0
Stream drops	0
Blocks processed	19562
Sequence gaps reported by the host script	1 (236 blocks missing)

If any are non-zero, what happened?

I havent looked into it, but I suspect initial synchronization for the single sequence gap.

4. Your AGC

	Value
How fast the gain comes down	8ms (lookahead - no spike observed)
How fast it goes back up	150ms to 95% of gain target (exponential)
Gain limits you used	clamped -20dB and +40dB
How often you update the gain	measure: once per block (125hz) update: linear interpolation across samples (8kHz)

Anything you tried that did not work?

Surprisingly, no. Not even a single hw misconfiguration. Pleasant debug time. Easy to find docs, have done this kind of thing before, and claude rocked it.

5. Your filter

What kind, what order, where the coefficients came from, and what it does to the signal.

Symmetrical FIR filter w/ 150hz skirt width, 60dB attenuation. Static memory, coefficients computed at runtime. Frankly I don't know this math. The bandpass is one low pass minus another, and we subtract their ideal impulse responses.

Apparently this filter type is called "A Kaiser-windowed-sinc linear-phase FIR bandpass, Type I."

Bessel is mentioned in the code because a Kaiser window is defined in terms of the bessel function.

I chose this filter for its linear group delay response, and because the CPU / memory usage was acceptable. This was one way for me to implement a filter that I could be confident relying on in downstream applications like ragefinding, because there would be no frequency dependent delay in reported signals. The delay is static and can be accounted for cleanly.

6. Real-time design

Which interrupts you used and what priority you gave them, and why:

DMA interrupts 1 / 2, for the ADC→ memory and the memory→ UART portions
Timer6 interrupt to trigger GPIO (there's also a hardware signal linked to ADC trigger from this peripheral but that's not an "interrupt" in the classic sense

Where does data cross from interrupt context into your main loop, and why is that safe?

DMA sets a volatile flag indicating that it is done with one or the other half of the capture_buffer. Safe because the only consumer is the main loop, basically. If the main loop slowed down we would detect an overwrite.

What happens if the serial link stops draining?

rb_write_all drops when there is not enough space. processed data would start being dropped on the floor until the UART sent enough bytes to make room for another frame.

7. What you would do next

With another day: Hardware filtering for aliased high frequencies. If I can't do HW filtering then I might increase the sample frequency of the whole system, though this increases resource usage.

The weakest part of what you built:

Hardware filtering for aliased high frequencies. If I can't do HW filtering then I might increase the sample frequency of the whole system, though this increases resource usage.

AGC would be weak for a voice application — could add things like holdover before ramping up quiet noise.

8. AI assistance

Which tools you used, and for what:

I used anthropic tooling for a lot of this; research about filter architectures, peripheral code, and even writing up DMA pipelines.

What they got wrong that you had to fix. This is the part we find most useful — be specific.

Nothing glaringly wrong, and I looked carefully. Most trouble from overzealous Claude trying to ram in features I did not need. Probably over-constrained on FIR unit tests. Claude and I worked well together esp. becuase I had an example repo to guide style + conventions.

9. Anything else